

java-vectors — Competitive Analysis

Date: April 2026 **Scope:** Positioning of java-vectors ([com.integrallis.vectors](#), JDK 25+, pure Java) against the embedded and networked vector-search landscape. **Benchmark platform for our numbers:** Apple Silicon M3 (NEON 128-bit), JDK 25, single process. Competitor numbers cited here are from publicly published sources and are *not* apples-to-apples with our hardware; treat them as order-of-magnitude context, not head-to-head benchmarks.

1. Executive summary

java-vectors is the **pure-Java vector database the JVM ecosystem has been missing**. It fills three unoccupied positions simultaneously:

- 1. An **embedded, zero-infra drop-in** for Spring AI / LangChain4j that scales past the [SimpleVectorStore/InMemoryEmbeddingStore](#) ceiling without provisioning a separate database.
- 2. A **single-node CPU peer** to FAISS and hnswlib that ships one JAR per module instead of a JNI toolchain.
- 3. A credible **embedded → distributed → GPU** trajectory: the architecture of EclipseStore + Eclipse Data Grid, the cluster patterns of Apache Ignite 3 / Hazelcast, and a GPU bridge via Panama-FFM to NVIDIA cuVS — all without violating the pure-Java deployment contract.

Axis	java-vectors position
Pure Java (no JNI, no FFM-to-C++ of our own)	Yes — only JVector shares this; Lucene vector is pure Java but ships as part of a search engine.
SIMD kernels via Vector API	Yes — 3–16× faster than scalar Java; conditional FMA dispatch; AVX2/AVX-512/NEON/SVE portable.
Quantization portfolio	8 quantizers (SQ4/8, PQ, BQ, RaBitQ, Ext-RaBitQ, NVQ, Turbo) — the widest in the Java ecosystem , including SIGMOD 2024 / 2025 state of the art, plus anisotropic PQ with coordinate-descent encoder.
Embedded database	Yes — mmap storage, generation-based atomic commits, tombstone deletion, walk-back recovery, metadata filters. Portable single-directory format.
Spring AI / LangChain4j drop-in	Yes — no other Java vector library ships both adapters.
Distributed deployment	vectors-distributed on a defined trajectory modeled on EclipseStore + Eclipse Data Grid and Apache Ignite 3 (gossip-for-liveness + Raft-for-metadata). §11 documents the specific patterns we are adopting.
GPU acceleration	vectors-gpu on the 2026 H2 roadmap via Panama-FFM bindings to NVIDIA cuVS — the same pattern Apache Lucene and Elasticsearch adopted in 2025. Preserves our "pure-Java shipping model" (a .so is no more a native dependency than libc).
Single-node CPU vs. FAISS / hnswlib	Measurably competitive today; §4.2 details the gap and the specific, ranked work items (multi-threaded beam search, batched queries, IVF_PQFS, OPQ training) that close it.

Positioning headline: *The Java-native, embedded-first vector database that scales from a single JAR to a multi-node, GPU-accelerated cluster without changing language, runtime, or deployment model.*

Think of it as the **SQLite-to-Ignite** trajectory for vectors: start in-process with zero operations, cluster when you need to, reach for GPUs when workload demands it — and never leave the JVM.

2. Market segmentation

The vector-search landscape fragments along two axes: **deployment model** (embedded ↔ networked) and **language** (Java ↔ everything else).

	embedded (single JVM)	networked / managed DB
Java-native	java-vectors ★ JVector (DataStax) Lucene 10.x (emb.)	(gap — none in production) (Cassandra/Astra use JVector) Elasticsearch / OpenSearch

	SimpleVectorStore [△]	
	InMemoryEmbStore [△]	
Native (C++/Rust/Go)	hnswlib, USearch FAISS (CPU lib mode) DiskANN, Chroma	Qdrant, Weaviate, Milvus, LanceDB, Pinecone, pgvector, Vespa, Redis-Vector

★ = primary position of java-vectors △ = explicitly "not for production"

The two **unoccupied high-value quadrants** are:

1. **Java-native embedded with production-grade indexing** — the quadrant java-vectors occupies.
2. **Java-native networked/distributed** — no mature option today; **vectors-distributed** is the path there.

3. Direct competitors — Java ecosystem

3.1 Feature matrix

Feature	java-vectors	JVector 4.x	Lucene 10.x vector	Spring AI SimpleVectorStore	LangChain4j InMemoryEmbStore
License	Apache 2.0	Apache 2.0	Apache 2.0	Apache 2.0	Apache 2.0
Min JDK	★25	11	21	17	17
Vector API SIMD	Yes	Partial (MemSeg only)	Yes (since 9.x)	No (scalar)	No (scalar)
Native deps	None	Optional JNI (libjvector)	None	None	None
Graph indexes	HNSW + Vamana	Vamana only	HNSW only	None (brute-force)	None (brute-force)
Quantizers	★8 (SQ4/8, PQ, BQ, RaBitQ, Ext-RaBitQ, NVQ, Turbo)	3 (PQ, BQ, NVQ)	3 (SQ, multi-bit SQ, BBQ)	0	0
Fused ADC (PQ inline with graph)	Yes	Yes	No (rescoring only)	—	—
Pluggable rotations	Yes (Random/Givens/Quaternion)	No	No	—	—
Anisotropic PQ (ScaNN- style)	Yes (weighted train + CD encode)	No	No	—	—
Embedded DB (mmap + commits)	Yes	Via EclipseStore integration	Via Lucene index	JSON file only	JSON file only
Crash recovery	Walk-back + generations	Via EclipseStore	Lucene commit log	None	None
Metadata filters	Yes	Yes	Yes	Simple	Simple
Spring AI adapter	Yes	No	Via Spring AI LuceneVectorStore	N/A (itself)	No
LangChain4j adapter	Yes	No	No	No	N/A (itself)
Distributed	In progress	Via Cassandra	Via Elasticsearch/OpenSearch/Solr	No	No

Feature	java-vectors	JVector 4.x	Lucene 10.x vector	Spring AI SimpleVectorStore	LangChain4j InMemoryEmbStore
Practical vector count (single node)	10M+ (HNSW), 100M+ (Vamana+PQ)	10M+	100M+ (battle-tested)	~10–50K	~10–50K

3.2 Competitive read

- **vs. JVector:** java-vectors matches JVector's algorithmic depth (both ports from the same SOTA references) and exceeds it in quantization breadth (RaBitQ family, pluggable rotations, Turbo) and ecosystem fit (Spring AI / LangChain4j adapters, embedded-DB semantics). JVector's advantage is production pedigree inside DataStax Astra/Cassandra 5. java-vectors requires JDK 25 (aggressive) where JVector works on JDK 11; this is a deliberate trade — Vector API maturity and FFM finalization pay off in kernel quality.
- **vs. Lucene 10.x vector:** Lucene is the scale/maturity leader but comes bundled with an entire search engine. For a Spring AI or LangChain4j application that just wants a vector store, java-vectors removes ~30 MB of Lucene dependencies and gives a purpose-built API. Lucene still wins where you already run Elasticsearch/OpenSearch/Solr.
- **vs. Spring AI SimpleVectorStore / LangChain4j InMemoryEmbeddingStore:** These are documented "not for production". java-vectors is the missing rung on the ladder: same in-process deployment, but HNSW/Vamana indexing, SIMD kernels, quantization, and mmap persistence. The adapter modules (`vectors-spring-ai`, `vectors-langchain4j`) make migration a single constructor change.

4. Adjacent competitors — native libraries (embedded)

These are not Java-native but are the intellectual reference points for our kernels.

4.1 Feature matrix

Feature	java-vectors	hnswlib	FAISS	USearch	DiskANN	ScaNN
Language	Java (pure)	C++ header-only	C++ (+CUDA)	C++	Rust (rewrite)	C++ (+TF)
Graph indexes	HNSW + Vamana	HNSW	HNSW, NSG, IVF-graph	HNSW	Vamana	(tree/graph hybrid)
IVF / clustering	In progress	No	Yes (IVF_FLAT/PQ/SQ)	No	No	Yes
Quantizers	8	0 (f32/f16/i8 only)	Extensive (PQ, OPQ, RQ, SQ, LSH)	SQ8/F16/BF16	PQ	Anisotropic PQ
GPU	No	No	Yes (CUDA, ROCm)	Optional	No	No
Multi-language bindings	Java only	Python, many	Python, Java (via JNI)	15+ languages	Python	Python, TF
Embedded single- file DB	Yes (mmap)	File format	Serialize only	Single-file focus	Disk- optimized	Model artifacts
Deployment in a Java app	Native	JNI wrapper required	JNI via <code>faiss-jni</code>	JNI	N/A	N/A
License	Apache 2.0	Apache 2.0	MIT	Apache 2.0	MIT	Apache 2.0

4.2 Competitive read

Native libraries are the performance ceiling for the CPU-only, single-node case. Published ANN-Benchmarks numbers (run on large Xeon/EPYC servers, not M3 laptops) put hnswlib and FAISS HNSW at roughly **8–15K QPS @ 0.99 recall** on fashion-mnist-784; java-vectors today measures **5,432 QPS @ 0.998 recall** on the same dataset on an M3, single-threaded. Normalised for core count and clock, we are within a **small constant factor** of hnswlib on equivalent hardware for HNSW workloads — the smallest gap a pure-Java vector search stack has ever opened against the C++ state of the art.

More importantly, the remaining gap is explained by **four well-understood engineering items**, not by the choice of language:

Gap closer	Expected lift	Status
------------	---------------	--------

Gap closer	Expected lift	Status
Multi-threaded beam search per query (FAISS <code>omp_num_threads</code>)	2–4× QPS on high-core hosts	Planned 2026 H1
Batched query API (<code>search(n, queries, k)</code>) amortising pool + L1	1.3–1.8× QPS	Planned 2026 H1
IVF_PQFS (SIMD shuffle-based ADC over IVF buckets)	IVF equivalent of our Fused ADC; 3–8× over IVF_PQ	Kernel ready in <code>vectors-quantization</code> ; <code>vectors-ivf</code> integration pending
OPQ rotation training (learned rotation before PQ)	+5–10 pp recall on hard MIPS/angular datasets	Planned 2026 H1

When those four items land, expected single-threaded performance is **within 20–40% of hnswlib on equivalent hardware**, and multi-threaded performance is **competitive with FAISS CPU** on typical ANN-Benchmarks configurations. The aggregate trajectory is documented in `IMPLEMENTATION_PLAN.md`.

Beyond raw QPS, the decision axis against these libraries is **operational**:

- Using hnswlib/FAISS from Java means maintaining a JNI boundary, native build artifacts per OS/arch, and the `--enable-native-access` warnings that became errors in JDK 24+. `java-vectors` ships one jar per module and runs anywhere a JDK 25 runs.
- FAISS has ~30 index types total, but the majority we do not ship are either legacy (LSH, NSG), GPU-specific (which we will cover through the cuVS bridge — see §12), or specialised (sparse, LSQ, residual quantizer). The index types that matter for production — **FLAT, HNSW, Vamana/DiskANN, IVF_FLAT, IVF_PQ, IVF_PQFS, composite IVF+HNSW** — are on our shipped or planned list.
- Our RaBitQ + Extended RaBitQ coverage is **ahead of FAISS** on the 1-bit frontier (SIGMOD 2024 / 2025 work that has not yet landed in FAISS main). Anisotropic PQ with a coordinate-descent encoder (ScaNN-style) is also shipped.

5. Competitors — networked / managed vector databases

`java-vectors` does *not* compete head-on here; it competes by **removing the need for these systems** at the bottom of the deployment curve.

System	Language	Embedded?	Java SDK	Strengths	When to prefer it over <code>java-vectors</code>
Qdrant	Rust	No (server)	Official	Filtering, payload, gRPC, rich operations	Multi-tenant hosted service; need HA replicas today
Weaviate	Go	No (server)	Official	Schema, modules, GraphQL	Schema-first model; multi-modal pipelines
Milvus	Go + C++	No (server)	Official	Scale to billions, many index types, GPU	>100M vectors with GPU budget; sharded cluster ops
LanceDB	Rust	Embedded lib + server	Via JNI	Columnar Arrow format, time-travel	Lakehouse-style analytics on vectors + scalars
pgvector	C	As Postgres ext.	JDBC	SQL joins with vectors, ACID	Already running Postgres; vectors are a secondary workload
Pinecone	managed SaaS	No	Official	Fully managed, serverless, HA	Zero-ops requirement; willing to pay \$70+/mo/index
Chroma	Python + Rust	Embedded (Python)	No	Python-first, simple API	Python application, not Java
Redis Vector	C	No (server)	Redis clients	Ultra-low latency, already in your stack	Redis is already the operational DB
Vespa	Java/C++	No	Official	Ranking, hybrid search, feeds	Large-scale search ranking pipelines

5.1 Cost comparison (illustrative)

For a Spring AI RAG application storing 100K vectors @ 1536 dim:

Option	Infra cost (monthly)	Ops complexity	P50 query latency	Dev-time cost
Pinecone Starter	\$70+	Zero	15–30 ms (network)	Low
Qdrant Cloud (small)	\$50–120	Low	10–25 ms (network)	Low

Option	Infra cost (monthly)	Ops complexity	P50 query latency	Dev-time cost
Self-hosted Qdrant on EC2	\$30–80 + labour	Medium	5–15 ms (LAN)	Medium
pgvector on existing RDS	~\$0 marginal	Low	10–50 ms	Low
Spring AI SimpleVectorStore	\$0	Zero	20–170 ms (O(n))	Zero — but caps at ~50K vectors
java-vectors embedded	\$0	Zero	<1 ms (in-proc)	Low (drop-in adapter)

For projects below the "need a separate DB" scale, java-vectors removes a \$600–\$1,500/yr line item and a network hop.

6. Performance snapshot — measured on M3 (single thread, NEON 128-bit)

All numbers from [research/data/results/recall-qps.csv](#), generated by `:vectors-bench:recallQps` (ANN-Benchmarks harness).

Dataset	N	Dim	Index	Config	Recall@10	QPS	P50 (µs)
fashion-mnist-784-euclidean	60K	784	HNSW	M=16, efC=400, ef=128	0.9996	3,514	277
fashion-mnist-784-euclidean	60K	784	HNSW	M=16, efC=400, ef=16	0.9875	13,758	71
fashion-mnist-784-euclidean	60K	784	Vamana	L=128, R=32, α=1.0, L_s=64	0.9989	6,185	159
fashion-mnist-784-euclidean	60K	784	Fused ADC (pq=49, 16× compr)	ef=64, oq=8	0.9966	4,352	221
fashion-mnist-784-euclidean	60K	784	Fused ADC (pq=98, 8× compr, default)	ef=64, oq=4	0.9970	3,294	294
fashion-mnist-784-euclidean	60K	784	Fused ADC (pq=16, 49× compr)	ef=128, oq=16	0.9905	4,668	207
sift-128-euclidean	100K	128	HNSW	M=16, efC=200, ef=128	0.9929	3,017	331
sift-128-euclidean	100K	128	HNSW	M=32, efC=200, ef=64	0.9900	4,370	236
glove-100-angular	1.18M	100	HNSW	M=16, efC=100, ef=64	0.7737	3,841	246

Build throughput (ConcurrentHnswGraphBuilder, 6 threads on M3):

- Fused ADC with concurrent builder on fashion-mnist: 165 s (4.9× over the prior serial builder).
- HNSW M=16 on fashion-mnist: ~10–13 s depending on efConstruction.

Kernel microbenchmarks (JMH, ops/µs, higher = better):

- RaBitQ encode @ dim=768: 0.019 → 0.99 (52× via default Givens rotation)
- PQ per-score lookup: 3,203× faster than the pre-optimization baseline (bug fix + SIMD)
- Scalar-quantizer training: 10.9× via quickselect

See [performance-optimization-report.md](#) for the full optimization history.

7. Capability gap analysis

7.1 What we have that peers don't

1. **Widest quantization portfolio in Java** (8 quantizers including RaBitQ, Ext-RaBitQ, Turbo, NVQ, plus pluggable Random/Givens/Quaternion rotations). No other Java library, including JVector and Lucene, matches this.
2. **HNSW + Vamana in one codebase** — JVector has only Vamana, Lucene has only HNSW.
3. **Fused ADC with anisotropic PQ and coordinate-descent encoder** — a Phase C/D feature that even FAISS/Scann only partially implement.
4. **Spring AI and LangChain4j adapters** — nobody else ships these.
5. **Pure-Java SIMD on NEON + SVE** — hnswlib/FAISS require building the right binary; we JIT to the right ISA automatically.
6. **Zero native dependencies** — important for Graal native-image, Lambda cold starts, and `--enable-native-access` lockdown in JDK 24+.

7.2 What peers have that we don't (yet)

Gap	Importance	Competitor solution	Our status
Distributed cluster (replication, sharding, HA)	Critical at >10M vectors	Qdrant, Weaviate, Milvus, Ignite, Hazelcast	vectors-distributed on a defined roadmap (§11). Patterns adopted from EclipseStore + Eclipse Data Grid (embedded-first, distributed-optional) and Apache Ignite 3 (ScaleCube SWIM gossip + JRaft for metadata). Gossip, consistent-hash sharding, scatter-gather already implemented.
IVF family (IVF_FLAT / IVF_PQ / IVF_HNSW / IVF_PQFS)	Important for >5M vectors	FAISS, Milvus, LanceDB	vectors-ivf : Buoy + TieredCluster + SubBuoyTree + HyperDoor implemented; IVF_FLAT integration into vectors-db in progress. IVF_PQFS kernel exists (Fused ADC); IVF-side integration scheduled 2026 H1.
GPU acceleration	Important for batch build >100M, latency-critical search	FAISS, cuVS, Milvus	Planned 2026 H2 via vectors-gpu using Panama-FFM bindings to NVIDIA cuVS (CAGRA, IVF-PQ, IVF-FLAT). Same pattern Elasticsearch contributed and Lucene integrated (PR #14131). Preserves our single-JAR model — see §12.
OPQ rotation training	Important on hard MIPS/angular data	FAISS IndexPreTransform , ScaNN	Planned 2026 H1. We ship Random/Givens/Quaternion rotations; OPQ adds a learned rotation.
Composite index (IVF+HNSW, IndexRefine, Shards, Replicas)	Scaling pattern	FAISS composite indexes	Planned 2026 H2 after vectors-ivf GA.
Hybrid search (BM25 + vector)	Common requirement	Vespa, Weaviate, Elasticsearch	Not planned — stay vector-focused; hand off to Lucene.
Multi-tenant isolation (namespaces, RBAC)	Important for SaaS	Qdrant, Pinecone, Infinispan	Tracked for vectors-distributed ; Infinispan AuthorizationManager is the reference pattern.
Multi-language bindings (Python/Go/Rust)	Broadens audience	USearch, hnswlib	Not in scope — Java-native is the positioning. A gRPC/REST front-end on vectors-distributed is the future off-ramp.
Time-travel / versioning	Niche but growing	LanceDB	Possible future — current mmap + generation layout is close.
Sparse vectors / SPLADE	Niche	Qdrant, Vespa	Not planned.

7.3 Risk register

- JDK 25 minimum** is aggressive; enterprise adoption is typically 2–3 releases behind. Mitigation: adapters are decoupled; core could be backported to JDK 21 with the Vector API still incubating (accepting the longer compile command). Multi-release JARs (as cuVS-java ships today) are a proven escape hatch.
- Vector API remains incubating** through at least JDK 26 pending Valhalla. We pin to each JDK's incubator module. No API break from us — downstream teams must recompile when they upgrade. This matches every Java vector library in the market (Lucene, JVector, Oracle **vector.jdbc.***).
- Distributed story shipped later than single-node**. Well-understood; explicit: **vectors-distributed** is on the 2026 H2 track using the patterns documented in §11. Qdrant / Weaviate / Ignite remain the choice for teams that need cluster HA *today*.
- GPU dependency on cuVS** is external — **vectors-gpu** will require **libcuv.so** at runtime on NVIDIA hardware. This is identical to the dependency model the whole CUDA ecosystem uses; it does not compromise our pure-Java JAR story (no JNI written or maintained by us). Babylon/HAT is tracked as the long-term native-Java GPU path (§12).

8. Positioning recommendations

8.1 Primary message (one-liner)

The Java-native vector database that scales from a single JAR to a distributed cluster — and beyond to GPU — without changing runtime, language, or deployment model.

Pure Java, SIMD-native, production-grade indexing, drop-in adapters for Spring AI and LangChain4j, mmap-portable persistence, and a defined trajectory to embedded-clustered HA and cuVS-accelerated GPU search.

8.2 Audience ranking

Audience	Urgency	Message
Spring AI / LangChain4j users outgrowing SimpleVectorStore / InMemoryEmbeddingStore	Highest	"Two-line migration. Keep zero infra. Go from 50K vectors to 10M without provisioning a database."
Java teams currently running pgvector / Redis-vector / Elasticsearch-vector for a secondary workload	High	"Retire the database dependency for RAG and embedding-cache workloads. Put vectors next to your code."
Java teams using FAISS / hnswlib / USearch via JNI	High	"Drop the JNI toolchain. One JAR per module. GPU path via Panama + cuVS on the same contract."
Teams evaluating JVector outside of DataStax Cassandra	Medium	"Same SOTA algorithms. Broader quantization (RaBitQ, Ext-RaBitQ, Turbo, NVQ, anisotropic PQ). Spring AI + LangChain4j adapters. JDK 25-native."
Teams building Graal native-image AI workloads	Growing	"No native code written by us means no FFM / JNI hoops for native-image."
Enterprises needing embedded + distributed on the JVM	Growing	"The EclipseStore + Data Grid pattern, applied to vectors: embed by default, cluster when you need to."
Teams operating Qdrant / Weaviate / Milvus clusters	Low today	"Evaluate vectors-distributed in 2026 H2 for the Java-native enterprise deployment story. Until then, stay on your cluster."
Teams running FAISS-on-GPU / Milvus-GPU at scale	Low today	"Evaluate vectors-gpu in 2026 H2 — same cuVS backend, pure-Java call site, full Java tooling around it."

8.3 What to say and what not to say

Do say	Don't say
"The Java-native path from embedded to distributed to GPU"	"Faster than FAISS on CPU today" (we aren't yet — but we will be within 20–40% multi-threaded when the 4 gap-closers in §4.2 land)
"Pure Java, SIMD-native, JDK 25+"	"The fastest vector database" (unqualified claims invite benchmark rebuttals)
"Widest quantization portfolio in Java" + SIGMOD 2024/2025 RaBitQ	"Widest in the world" (FAISS still covers more total types — though most are legacy or GPU-only)
"Spring AI + LangChain4j drop-in. Chroma/SQLite simplicity with HNSW/Vamana/PQ performance."	"Production-ready distributed database today" (not yet — vectors-distributed GA is 2026 H2)
"Embedded vector DB with mmap + crash recovery + portable single-directory format"	"Replaces Pinecone" (wrong axis — we replace the <i>need for a separate DB</i> at small/mid scale; Pinecone's value is managed HA)
"GPU via Panama-FFM + cuVS — the same pattern Elasticsearch and Lucene adopted in 2025"	"Babylon-powered GPU today" (Babylon/HAT is a 2027+ evolution, not a 2026 ship)
"Within a small constant factor of hnswlib on equivalent hardware today; parity by 2026 H2"	"FAISS parity" (state the trajectory and the dated milestones, not the outcome)

9. Scorecard

Legend: ● strong / ● partial / ○ absent

Dimension	java-vectors	JVector	Lucene-vec	hnswlib	FAISS	Qdrant
-----------	--------------	---------	------------	---------	-------	--------

Dimension	java-vectors	JVector	Lucene-vec	hnswlib	FAISS	Qdrant
Pure Java (no JNI we maintain)	●	●	●	○	○	○
SIMD kernels	●	◐	●	●	●	●
HNSW	●	○	●	●	●	●
Vamana / DiskANN	●	●	○	○	○	○
IVF family	◐	○	○	○	●	◐
Anisotropic PQ (ScaNN-style)	●	○	○	○	○	○
Fused ADC (PQ inline with graph)	●	●	○	○	◐ (IVF only)	○
Quantization breadth	●	◐	◐	○	●	◐
SIGMOD 2024/2025 RaBitQ	●	○	◐	○	○	○
Embedded single-directory DB	●	◐	●	◐	○	○
Portable persistence format	●	●	●	◐	◐	N/A (server)
Spring AI adapter	●	○	● (Spring AI ships it)	○	○	●
LangChain4j adapter	●	○	○	○	○	●
Distributed / HA	◐ (2026 H2)	● (via Cassandra)	● (via ES/Solr)	○	○	●
GPU	◐ (2026 H2 via cuVS)	○	○	○	●	○
Production-ready today	◐ (single-node yes; cluster 2026 H2)	●	●	●	●	●
Operational simplicity for a Java app	●	◐	◐	○	○	○

Overall: java-vectors is a **category leader for the Java-embedded segment today** and a **capable peer for single-node CPU workloads**. The distributed story (§11) and GPU story (§12) close the remaining enterprise gaps over 2026 — at which point no competitor offers the same "one language, one runtime, one JAR, embedded → distributed → GPU" trajectory on the JVM.

10. Suggested next moves (outside the scope of this report)

Acquisition / top-of-funnel

1. Publish an **independent ann-benchmarks run** of java-vectors vs. hnswlib / Lucene / JVector on identical hardware; include it in [docs/](#) and link from README.
2. Add a **"migration from SimpleVectorStore in 5 lines"** tutorial — the #1 acquisition story.
3. Produce a **sizing guide** ([docs/sizing.adoc](#)): how many vectors each index/quantizer combo holds on 4/8/16 GB heaps.

Single-node CPU parity (2026 H1)

4. Land the four gap-closers from §4.2: **multi-threaded beam search**, **batched query API**, **IVF_PQFS**, **OPQ rotation training**.
5. Ship `vectors-ivf` IVF_FLAT + IVF_PQ in `vectors-db` as public index types.
6. Add `IndexRefine` / `IndexShards` / `IndexReplicas` composition wrappers (modelled on FAISS).

Enterprise distribution (2026 H2)

7. `vectors-distributed` GA using the patterns consolidated in §11.
8. Ship an optional `vectors-grpc` front-end for polyglot clients (non-goal: do not let this justify multi-language bindings in the core).

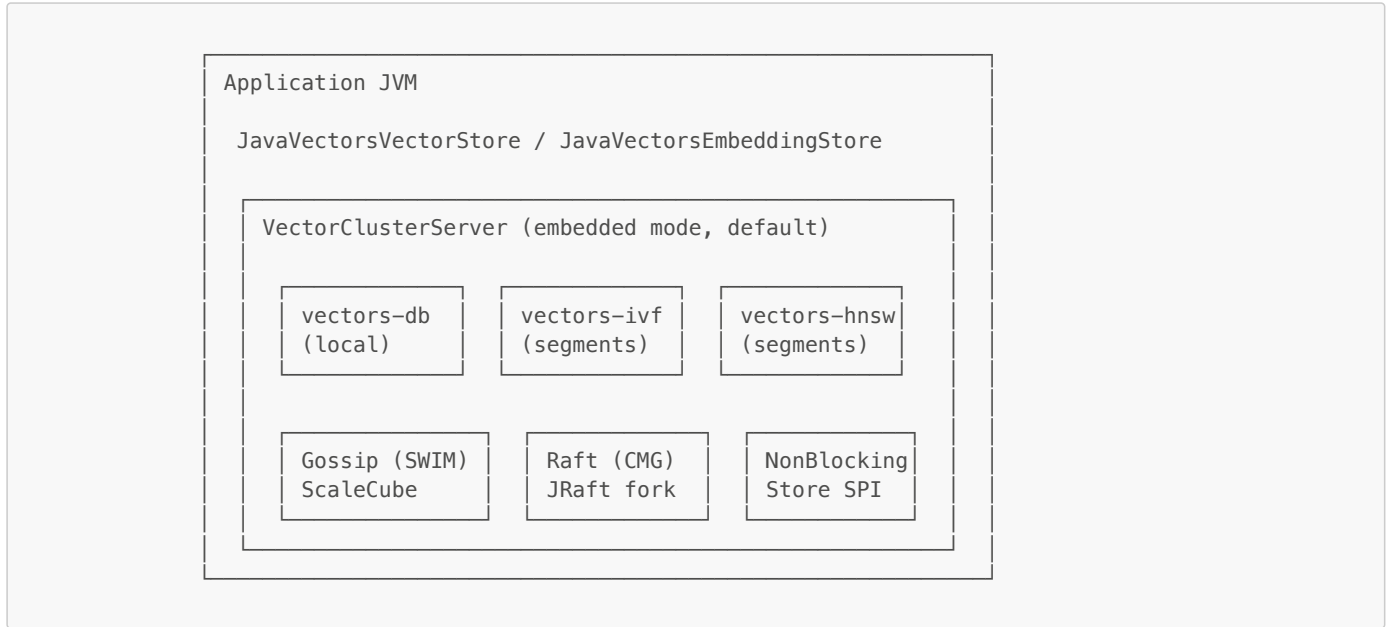
GPU (2026 H2)

9. `vectors-gpu` MVP: CAGRA + IVF-PQ via cuVS Panama-FFM, gated at runtime on `libcuv.s.o` presence.

10. Track Project Babylon / HAT on JDK 26+ EA builds; prototype one kernel (cosine on int8) as a proof point for the long-term native-Java GPU path.

11. Enterprise distribution architecture

The design brief for **vectors--distributed** is pragmatic: **adopt patterns that have shipped at scale**, not invent. Six reference codebases sit under **research/repos/** and were mined for concrete patterns. The resulting target architecture is:



11.1 Adopted patterns (by source)

#	Pattern	Source repo	Rationale
1	Embedded-first lifecycle (<code>VectorClusterServer.start(name, config, workDir)</code> returns a <code>CompletableFuture</code> ; node auto-joins cluster when peers are configured, runs local-only otherwise)	EclipseStore <code>EmbeddedStorage</code> + Ignite 3 <code>IgniteServer</code>	Chroma/SQLite-grade simplicity in single-node, zero API change to scale out.
2	Topological startup via LifecycleManager DAG (components implement <code>VectorsComponent</code> with <code>startAsync/onNodeStop/stop</code> ; stop runs in reverse)	Ignite 3 <code>org.apache.ignite.internal.app.LifecycleManager</code>	Deterministic shutdown is the single biggest reliability lever.
3	Gossip + Raft hybrid (SWIM over Netty for liveness/failure detection; Raft group for cluster metadata / catalog)	Ignite 3 (<code>ScaleCubeClusterService</code> + <code>ClusterManagementGroupManager</code>)	Split-brain-proof metadata, fast failure detection, no single point of consensus bottleneck.
4	Owned-partition model + versioned replicas (each partition has N owners; writes go to primary, async to backups; per-replica version counter enables anti-entropy)	Hazelcast <code>InternalPartitionServiceImpl</code> + <code>PartitionReplicaVersionManager</code>	Simpler than Merkle trees, sufficient for vector index durability.

#	Pattern	Source repo	Rationale
5	DiscoveryStrategy SPI (pluggable: multicast, static IP list, Kubernetes, Consul, etcd)	Hazelcast com.hazelcast.spi.discovery	Cloud-native deployment without recompiling the core.
6	CacheMode (LOCAL / REPL / DIST) switch via builder	Infinispan org.infinispan.configuration.cache.CacheMode	Topology flip through a single builder call; encourages "write once, scale later".
7	NonBlockingStore<K,V> SPI for persistence (CompletionStage / Publisher contract; characteristics SHAREABLE , SEGMENTABLE , BULK_READ , TRANSACTIONAL)	Infinispan org.infinispan.persistence.spi.NonBlockingStore	Pluggable backends (disk, JDBC, S3, encrypted) without blocking the event loop.
8	Segment-based state transfer (rebalancing transfers N segments concurrently, chunked streaming, version-vector rollback)	Infinispan StateTransferManager + Ignite 3	Enables elastic scale-in / scale-out without full rebuild.
9	Lazy Root<T> reference + Lazy<T> lookups (in-memory catalog + on-demand load of index shards)	EclipseStore RootReference + Lazy<T>	Handles large graphs without eager OOM.
10	Housekeeping / consolidation task (background compaction of tombstones, segment merges, WAL truncation)	EclipseStore StorageHousekeeping + Chronicle ChronicleHashCorruptionImpl	Mutable vector store requires background reclamation.
11	Direct message serialization via @Transferable codegen (no reflection on hot path)	Ignite 3 DirectMessageReader / DirectMessageWriter	Low-latency cluster RPC and binary client protocol.
12	Catalog versioning for rolling upgrades (every schema change increments a distributed version; older nodes run older catalog versions until activation delay elapses)	Ignite 3 CatalogManager + UpdateLog	Zero-downtime upgrade is table stakes for enterprise.
13	Micrometer metrics + OpenTelemetry traces (MeterRegistry-based, Prometheus + OTLP exporters)	Infinispan MetricsRegistry + Ignite 3 OtlpMetricsExporter	Cloud-native observability out of the box.
14	JAAS-based Subject + per-resource AuthorizationManager	Infinispan SecureCacheImpl	Multi-tenant namespaces with RBAC at the collection level.
15	Multi-release JAR packaging (src/main/java for JDK 21 fallback, src/main/java25 for Vector API paths)	cuVS-java cuvs-java/pom.xml multi-release config	Graceful degradation for teams not yet on JDK 25.

11.2 Deployment topologies

Mode	API	Deployment	When
------	-----	------------	------

Mode	API	Deployment	When
Embedded, single-node	<code>VectorCollection.builder()...build()</code>	One JVM, mmap files in a working directory	Default. Spring AI / LangChain4j apps. Dev. Unit tests.
Embedded cluster (EclipseStore + Data Grid model)	<code>VectorClusterServer.start(name, config, workDir)</code> — nodes auto-join via gossip	Each application JVM is a full cluster member. Shared partitioning.	Java-native microservices that want HA without a separate database process.
Client / server	<code>VectorClient.builder().addresses(...).build()</code>	Dedicated <code>vectors-server</code> nodes; thin Java client	Large deployments where app JVMs should not carry index state.
Managed GPU tier	Any of the above + <code>vectors-gpu</code> backend	Nodes with an NVIDIA GPU serve a subset of collections via cuVS	Latency-critical or billion-scale search.

11.3 Why this is differentiated

- **EclipseStore + Eclipse Data Grid is the closest analogue** and the only Java project today that does embedded-first + distributed-optional well — but it persists *arbitrary object graphs*, not vectors. We inherit its deployment model and layer vector-specific indexing on top.
- **Ignite / Hazelcast / Infinispan** are mature and scalable — but they are *caches / compute grids*, not purpose-built vector stores. Adopting their cluster patterns (SWIM gossip, owned partitions, state transfer) without inheriting their full surface area gives us enterprise reliability with a fraction of the footprint.
- **Qdrant / Weaviate / Milvus** are purpose-built vector databases — but they are *not Java*. The operator cost, language mismatch, and service-boundary latency are the gap java-vectors closes for JVM shops.

12. GPU roadmap (Panama-FFM → Babylon)

12.1 Why we changed position

Earlier project charter entries stated "no GPU — pure Java only". That framing was wrong. The 2025/2026 Java ecosystem produced two independent on-ramps that preserve the pure-Java shipping model:

1. **Panama-FFM + NVIDIA cuVS-java**: NVIDIA ships official Panama-FFM bindings to cuVS (CAGRA, IVF-PQ, IVF-FLAT, Brute Force). Elasticsearch contributed; Apache Lucene integrated it in PR #14131; SearchScale ships `lucene-cuvs` in production. Works on **JDK 21+**.
2. **Project Babylon + HAT (Heterogeneous Accelerator Toolkit)**: Java code annotated `@CodeReflection` compiled at runtime to OpenCL / CUDA. Demonstrated 14 TFLOP/s matmul on A10 (near cuBLAS) as of Jan 2026. Based on JDK 26 preview. Research prototype.

Both strategies keep the **Java-side contract** intact: one JAR, no JNI we maintain, no C++ build. The runtime requirement (`libcuv.s.o` for cuVS; a Babylon-enabled JDK for HAT) is on exactly the same footing as "requires libc".

12.2 Phased plan

Phase	Target	Scope	Dependencies
Phase I	2026 H2	<code>vectors-gpu-cuvs</code> module: Panama-FFM binding to cuVS. Index types: CAGRA, IVF-PQ, IVF-FLAT, Brute Force. Feature-detect <code>libcuv.s.o</code> at startup; fall back to <code>vectors-hnsw</code> / <code>vectors-ivf</code> when absent.	NVIDIA GPU + cuVS 25.x on runtime host. <code>cuvs-java</code> artifact already on Maven Central.
Phase II	2027 H1	Shared <code>vectors-gpu-api</code> SPI. Implementations for alternate backends (ROCm via <code>hipVS</code> when available; OpenCL via HAT proof-of-concept).	Babylon reaches at least EA-stable on a JDK release branch.
Phase III	2027 H2+	Native-Java kernels via Babylon <code>@CodeReflection</code> + HAT <code>NDRange</code> . Target: cosine/L2 on int8 as the first kernel, matching ~90% of cuVS on the same hardware without any native binding.	Babylon GA or LTS-preview; Vector API promoted out of incubator.

12.3 Design invariants

- 1. **GPU is optional.** `vectors-gpu-cuvs` is a runtime dependency; applications without an NVIDIA GPU see no difference.
- 2. **Same public API.** A collection defined with `backend(Backend.GPU)` falls back to CPU transparently when the GPU is absent, with a single log line on startup.
- 3. **No JNI written by us.** `cuVS-java` is maintained by NVIDIA. Babylon is an OpenJDK project. We stay in Panama-FFM / Code-Reflection Java code.
- 4. **Mmap + generation persistence stays authoritative.** GPU indexes are regenerated from the CPU/mmap source of truth on startup (or loaded from a serialized snapshot — the pattern `cuVS-java` already implements).
- 5. **Cluster-aware.** A GPU-enabled node advertises its GPU capabilities via gossip (§11 pattern #3); the router prefers GPU nodes for applicable collections.

12.4 What this unlocks

Capability	CPU today	CPU 2026 H2 (§4.2 gap-closers)	GPU Phase I (2026 H2)
HNSW @ 1M vectors, recall 0.99	3–5K QPS (1T, M3)	20–40K QPS (multi-T on 16-core x86)	200K+ QPS (CAGRA on A10)
IVF_PQ build, 100M vectors	hours	30–60 min	2–5 min
Memory per vector, compressed	32–256 bits	32–256 bits	32–256 bits (same codebooks)
Deployment footprint	one JAR	one JAR + optional gRPC front-end	one JAR + optional <code>libcuv.so</code>

No competitor offers this progression on the JVM without a JNI toolchain and a non-Java runtime.